IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Un botones virtual que aprende probando

Teoría, práctica y, al final, material de consulta

Sprint 9 · S09_04 · teoría breve (15 min el 16/11), 25 min de práctica en clase y material de consulta

LA IDEA · APRENDER PROBANDO

La tercera familia: premios y castigos

Supervisado

Hay respuesta correcta (**cancelada**): el modelo aprende a predecirla.

No supervisado

No hay respuesta: el algoritmo descubre grupos o ejes (S09_01 a S09_03).

Por refuerzo

No hay respuesta correcta: un **agente** prueba, recibe **recompensas** y aprende qué le conviene.

Pieza	En el hotel
Agente y entorno	el botones virtual y el pasillo del hotel
Estado	la casilla donde está (20 posibles)
Acción	arriba, abajo, izquierda o derecha
Recompensa	−1 por paso, −5 en obras, +10 al llegar a la habitación
Política	qué hacer en cada casilla
Episodio	una partida: de recepción a la habitación (o 50 pasos)

EL ENTORNO

El pasillo del hotel: 4 × 5 casillas



Salida: casilla 0 (recepción) · el número pequeño es el estado

Tres pasos de demostración

derecha → casilla 1 · derecha → casilla 2 · abajo → **choca con la pared y sigue en la 2**. Cada paso: **−1**.

Dos caminos de 7 pasos

Por las obras: −7 − 5 + 10 = **−2**.

Rodeándolas: −7 + 10 = **+3**.

Meta de la partida: llegar a la habitación con la mayor recompensa total, no solo llegar.

LA TABLA Q

Q: cuánto espero ganar si hago esto aquí

Q[estado, acción] empieza en **ceros** (20 filas × 4 columnas). Tras cada paso se corrige un poco hacia lo que **ahora** se cree que vale esa acción.

Ejemplo de una actualización	
Q[s, a] antes	2,00
recompensa r del paso	−1
mejor futuro: max Q[s']	5,00
objetivo = r + gamma · max Q[s'] (gamma 0,95)	3,75
Q nuevo = 2,00 + alpha · (3,75 − 2,00) (alpha 0,2)	2,35

alpha (0 a 1)

Velocidad de aprendizaje: alto se cree lo último; bajo cambia despacio.

gamma (0 a 1)

Cuánto importa el futuro. Con 0 solo mira la recompensa inmediata.

EXPLORAR O EXPLOTAR

A veces, probar algo al azar

epsilon	Qué hace el agente	Consecuencia
0,0	siempre la mejor acción que conoce	nunca descubre caminos mejores
0,2 (por defecto)	1 de cada 5 pasos, al azar	aprende y sigue explorando
1,0	siempre al azar	nunca aprovecha lo aprendido

Epsilon-greedy: casi siempre lo mejor conocido, y de vez en cuando una prueba.

SÍNTESIS DEL MÓDULO 3

¿Qué algoritmo para qué problema?

Tengo...	Familia	Algoritmos del curso
Un número que predecir	Regresión	Lineal y polinómica, Ridge y Lasso, árboles, Random Forest, boosting
Una categoría (sí o no, tipo)	Clasificación	Logística, k-NN, Naive Bayes, árboles, SVM, boosting
Datos sin etiqueta	No supervisado	Clustering para agrupar, PCA para reducir
Gustos de usuarios y productos	Recomendación	Popularidad, contenido, filtrado colaborativo
Decisiones en secuencia con premios	Refuerzo	Q-learning

Empieza por lo simple (regresión lineal o logística, un árbol) y complica solo si la validación lo justifica.

PRÁCTICA · CUADERNO S09_04

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Preparar las herramientas	Clase
Ejemplo	El entorno del hotel y tres pasos de prueba	Clase
Ejercicio 1	La regla de actualización de Q	Clase
Ejercicio 2	La función epsilon-greedy	Clase
Ejemplo	Entrenar al agente y ver la curva de aprendizaje	Clase
Ejercicio 3	La política y el camino aprendido (con el mapa de flechas)	Casa
Ejercicio 4	Con pasos gratis, ¿ayuda explorar?	Casa
Mini-reto	Obras a −20: cambiar la recompensa	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 4 y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Regla de actualización de Q	9 y 10
2	Epsilon-greedy	11 y 12
Ejemplo	Entrenar al agente y curva	13
3	La política y el camino	14 y 15
4	Pasos gratis y exploración	16 y 17
Mini-reto	Obras a −20	18 y 19

CONSULTA · EJERCICIO 1

La regla de actualización de Q (ejercicio 1)

```
def actualizar_q(Q, s, a, r, s2, alpha, gamma):
    objetivo = r + gamma * Q[s2].max()         # lo que ahora crees que vale
    Q[s, a] += alpha * (objetivo - Q[s, a])    # avanza una fracción alpha
```

1,3

Q[0, 3] con r = −1, max Q[5] = 4, alpha 0,5, gamma 0,9

objetivo = −1 + 0,9 × 4 = 2,6 → se avanza la mitad desde 0: 1,3.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la regla de Q (ejercicio 1)

Q[s, a] += alpha * (objetivo - Q[s, a])

Solo se modifica **una casilla**: la del estado y la acción que se acaban de probar. No la fila entera.

Q[s2].max()

El **mejor valor** que se espera desde el estado siguiente: la mirada al futuro.

Cómo sale 1,3

objetivo = −1 + 0,9 × 4 = **2,6**; con **alpha = 0,5** se avanza la mitad desde 0: 0,5 × 2,6.

Otros valores

Con **alpha = 1** se queda con **2,6** (se cree lo último). Con **gamma = 0** el objetivo es **−1**: no mira al futuro y no aprende a llegar a una meta lejana.

CONSULTA · EJERCICIO 2

La función epsilon-greedy (ejercicio 2)

```
def elegir_accion(Q, s, epsilon, rng):
    if rng.random() < epsilon:
        return int(rng.integers(Q.shape[1]))    # explorar: una acción al azar
    return int(np.argmax(Q[s]))                 # explotar: la mejor de la fila s
```

siempre la mejor

epsilon = 0

≈ 20 % al azar

epsilon = 0,2

Con **epsilon = 1** explora siempre las 4 acciones al azar.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer epsilon-greedy (ejercicio 2)

if rng.random() < epsilon

Decide **si explorar**: con probabilidad **epsilon**, una acción al azar; con **1 – epsilon**, la mejor conocida.

rng como argumento

Con el mismo generador y semilla se repiten **exactamente las mismas decisiones**: es reproducible.

rng.integers(Q.shape[1])

Cubre las **4 acciones**. Si la comparación **<** se invierte, siempre explora.

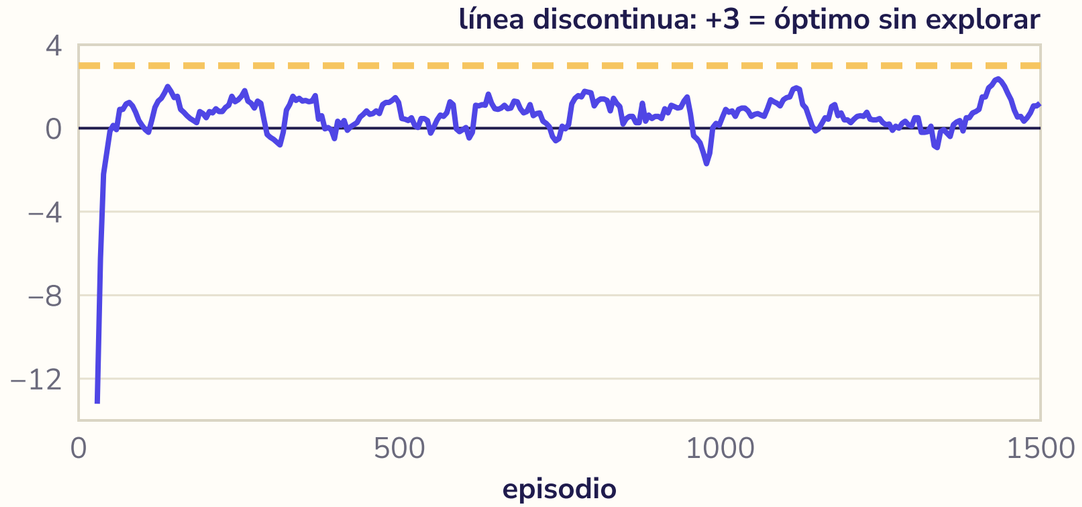
np.argmax(Q[s])

La mejor acción de la **fila del estado s**. Con **epsilon = 0,2**, ≈ **20 %** de las acciones son aleatorias.

CONSULTA · EJEMPLO · ENTRENAR AL AGENTE

1500 partidas y la recompensa mejora

Recompensa total por episodio (media móvil de 30)



−8,0 → 0,9

Media de los **primeros 50** y los **últimos 50**.

Desde el episodio 26

La política **sin explorar** ya es la óptima: **+3**.

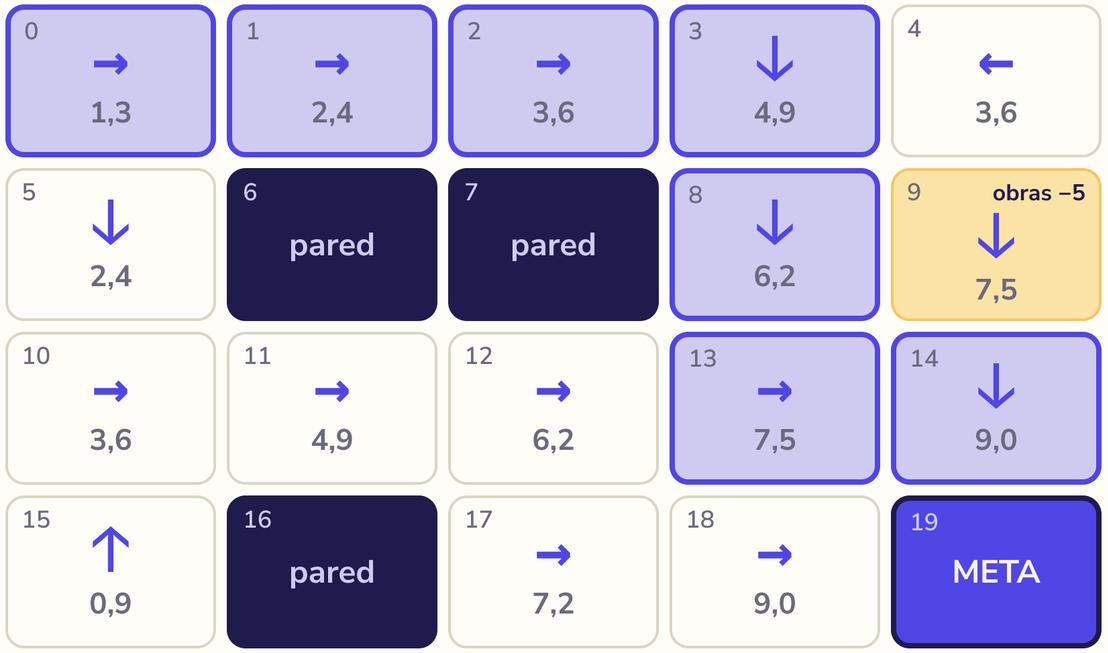
¿Por qué no llega a +3?

Con epsilon 0,2, 1 de cada 5 pasos es al azar.

```
Q, recompensas = entrenar()    # 1500 episodios: alpha 0,2 · gamma 0,95 · epsilon 0,2
```

CONSULTA · EJERCICIO 3

La política y el camino aprendido (ejercicio 3)



Flecha = mejor acción · número = valor de la casilla (max Q) · recuadro = camino aprendido

```
politica = np.argmax(Q, axis=1)

camino = [0, 1, 2, 3, 8, 13, 14, 19]
```

7 pasos y +3

Evita las obras (casilla 9) y llega a la habitación (casilla 19).

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la política (ejercicio 3)

np.argmax(Q, axis=1)

Una acción **por estado** (cada fila de Q). Con **axis=0** saldría una acción por columna.

El bucle del camino

entorno.step(politica[s]) hasta que **fin** sea verdadero (máximo 30 pasos): empieza en el estado 0 y llega al 19.

[0, 1, 2, 3, 8, 13, 14, 19]

7 pasos y +3: el más corto posible, **sin pisar las obras** (estado 9), porque pisarlas cuesta −5 más.

Mapa de flechas

Primera fila → → → ↓ ←. El valor sube hacia la meta (recepción ≈ 1,3; junto a la meta ≈ 9): **quedan menos pasos** que pagar.

CONSULTA · EJERCICIO 4

Con pasos gratis, ¿ayuda explorar? (ejercicio 4)

```
_, rec_eps0 = entrenar(epsilon=0.0, coste_paso=0.0)
_, rec_eps03 = entrenar(epsilon=0.3, coste_paso=0.0)
media_final_eps0 = rec_eps0[-100:].mean()                    # 0.0
media_final_eps03 = rec_eps03[-100:].mean()                  # 9.1
explorar_ayuda = bool(media_final_eps03 > media_final_eps0)  # True
```

Entrenamiento (1500 episodios)	Recompensa media de los últimos 100
pasos gratis (**coste_paso=0**), epsilon 0,0	0,0
pasos gratis (**coste_paso=0**), epsilon 0,3	9,1
coste −1 por paso (por defecto), epsilon 0,0	3,0
gamma=0 (sin mirar al futuro)	−49,5

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer el experimento (ejercicio 4)

rec_eps0[-100:].mean()

La media de los **últimos 100 episodios**: lo que ha aprendido al final del entrenamiento.

entrenar(...) devuelve dos cosas

Q y las **recompensas**: aquí se guarda la segunda (_ descarta Q).

0,0 frente a 9,1

Con pasos gratis y sin explorar **nunca llega a la meta** (empieza con Q en ceros y repite siempre la misma acción): no cobra ninguna recompensa.

Con coste por paso y gamma = 0

Con coste −1, hasta el que no explora aprende (chocar sale caro: **exploración implícita**). Con gamma = 0: **−49,5**, no aprende a llegar.

CONSULTA · MINI-RETO

Obras a −20: cambiar la recompensa (mini-reto)

```
class EntornoObras(EntornoHotel):                    # copia con otro nombre
    ...  # en step(): pisar las obras cuesta -20 en lugar de -5
Q_obras, _ = entrenar(clase_entorno=EntornoObras)    # nuevo parámetro de entrenar
politica_obras = np.argmax(Q_obras, axis=1)
```

Cambio de reglas en las obras	Camino aprendido (sin explorar)	Recompensa	¿Llega?
−5 (las reglas originales)	rodea las obras, 7 pasos	+3	sí
−20 (reclamaciones)	el mismo camino	+3	sí
+5 (una «recompensa» por pasar)	llega a las obras y **se queda en ellas**	+180	no

Una recompensa mal diseñada produce un comportamiento que no queremos.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el diseño de la recompensa (mini-reto, casa)

−20 en total −21

Pisar las obras debe costar **−20 más el −1 del paso**, no −25: se **sustituye** la penalización de −5.

entrenar y el entorno

Si **entrenar** sigue usando **EntornoHotel**, no se ve el cambio: hay que **pasarle la clase** nueva (**env = clase_entorno(coste_paso)**).

Obras a −20

La política sigue siendo **[0, 1, 2, 3, 8, 13, 14, 19]**: con −5 ya las evitaba, y con −20 con más motivo.

¿Y con +5?

El agente aprende a **ir a las obras y quedarse allí** (sigue sumando) y **no llega nunca a la meta**. Hace lo que se **premia**, no lo que queremos.